Ejercicio 6 | Polimorfismo via interfaces

Analisis y diseno - prototipo de consola con servicios simulados

Requisitos

Área	Requisito funcional	Responsable	Criterio de aceptación
Carga inicial	Instanciar al menos 10 integraciones variadas al iniciar	ControladorIntegraciones	Catálogo contiene 10 objetos en UNA lista ArrayList<IntegracionAPI>
Listar	Mostrar identificador, proveedor, tipo, estado de conexión y latencia de todas las integraciones	ControladorIntegraciones / Principal	Se visualizan las 10 conexiones y su estado
Buscar	Buscar por identificador y mostrar parámetros de configuración	ControladorIntegraciones	Encontrada: mostrar configuración; no encontrada: mensaje
Ordenar	Ordenar catálogo ascendentemente por latencia promedio (ms)	IntegracionAPI / ControladorIntegraciones	Comparable y Collections.sort sobre la misma lista
Transacciones	Simular cobro y reintegro únicamente en servicios Transaccional	Transaccional / APIPago	Reintegro a logística se ignora sin lanzar excepción
Documentos	Simular generación de XML FEL y guías de envío	Documentable / APIFacturacion / APILogistica	Se obtiene texto de documento simulado
Auditoría	Simular consulta de registro de operaciones en integraciones compatibles	Auditable / implementaciones	Polimorfismo mediante parámetro Auditable
Extensibilidad	Agregar otro proveedor instanciando clase concreta existente sin modificar motor central	ControladorIntegraciones	Nueva instancia se agrega a la misma lista
Menú	Menú de consola con listar, buscar, ordenar, cobro, reintegro, documento, auditoría y salir	Principal	Opciones válidas e inválidas se manejan sin terminar abruptamente
Restricciones	No usar while(true)+break; ninguna salida System.out.println fuera de Principal	Principal	Salida de texto centralizada en Principal
Una lista	Mantener una sola lista polimórfica de ítems; sin otras listas de integraciones	ControladorIntegraciones	Campo único ArrayList<IntegracionAPI>
Simulación	Datos, latencias y operaciones son ficticios; no consumir servicios reales	Todas	No requiere credenciales, HTTP ni API keys

Clases

Clase / interfaz	Tipo / capa	Responsabilidad	Implementa	Relaciones
IntegracionAPI	Clase abstracta / Modelo	Datos comunes; consulta estado y configuración; orden natural por latencia	Comparable<IntegracionAPI>	Superclase de los tres tipos
APIPago	Clase concreta / Modelo	Simula cobros, reintegros y auditoría	Transaccional, Auditable	extends IntegracionAPI
APILogistica	Clase concreta / Modelo	Simula rastreo, cotización, guía de envío y auditoría	Documentable, Auditable	extends IntegracionAPI
APIFacturacion	Clase concreta / Modelo	Simula certificación FEL, XML y auditoría	Documentable, Auditable	extends IntegracionAPI
Transaccional	Interfaz	Contrato de cobro y reintegro	—	Solo APIPago
Documentable	Interfaz	Contrato de generación de documentos	—	APILogistica y APIFacturacion
Auditable	Interfaz	Contrato de consulta de auditoría	—	Los tres tipos
ControladorIntegraciones	Controlador	Única lista polimórfica; carga, busca, ordena, despacha según interfaz	—	Usado por Principal
Principal	Vista / driver	Único System.out.println, Scanner, menú y formato de resultados	—	Usa ControladorIntegraciones

Atributos

Clase	Visibilidad	Atributo	Tipo	Propósito	Inicialización / validación
IntegracionAPI	-	id	String	Identificador único	Constructor; no vacío
IntegracionAPI	-	proveedor	String	Nombre visible	Constructor; no vacío
IntegracionAPI	-	endpoint	String	URL ficticia de servicio	Constructor; no vacío
IntegracionAPI	-	conectado	boolean	Estado de conexión simulado	Constructor
IntegracionAPI	-	latenciaMs	int	Latencia promedio en ms	Constructor; >= 0
APIPago	-	moneda	String	Moneda de pago	Constructor; no vacío
APILogistica	-	zonaCobertura	String	Área de entregas	Constructor; no vacío
APIFacturacion	-	nitCertificador	String	NIT de certificador ficticio	Constructor; no vacío
ControladorIntegraciones	-	integraciones	ArrayList<IntegracionAPI>	Única lista polimórfica de ítems	Inicializada en constructor
Principal	-	scanner	Scanner	Entrada por consola	Inicializado en main o constructor; NO lista

Métodos

Clase / interfaz	Visibilidad	Firma / parámetros	Retorno	Propósito y resultado	Validación / manejo
IntegracionAPI	+	IntegracionAPI(String id, String proveedor, String endpoint, boolean conectado, int latenciaMs)	Constructor	Inicializa atributos comunes	Rechaza id vacío y latencia negativa
IntegracionAPI	+	getId()	String	Consulta atributo encapsulado	—
IntegracionAPI	+	getProveedor()	String	Consulta atributo encapsulado	—
IntegracionAPI	+	getEndpoint()	String	Consulta atributo encapsulado	—
IntegracionAPI	+	getConectado()	boolean	Consulta atributo encapsulado	—
IntegracionAPI	+	getLatenciaMs()	int	Consulta atributo encapsulado	—
IntegracionAPI	+	setEndpoint(String valor)	void	Actualiza configuración / estado	Rechaza endpoint nulo o vacío
IntegracionAPI	+	setConectado(boolean valor)	void	Actualiza configuración / estado	Acepta true o false; no valida latencia
IntegracionAPI	+	setLatenciaMs(int valor)	void	Actualiza configuración / estado	Rechaza latencia negativa
IntegracionAPI	+	getTipo()	String	Método abstracto; cada subtipo define su categoría	—
IntegracionAPI	+	getConfiguracion()	String	Devuelve datos comunes y específicos; override en subclases	—
IntegracionAPI	+	compareTo(IntegracionAPI otra)	int	Comparable: latencia ascendente; desempate por id	—
IntegracionAPI	+	toString()	String	Resumen legible de integración	—
IntegracionAPI	+	equals(Object obj)	boolean	Identidad por id	—
IntegracionAPI	+	hashCode()	int	Consistente con equals	—
APIPago	+	APIPago(String id, String proveedor, String endpoint, boolean conectado, int latenciaMs, String moneda)	Constructor	Invoca super y almacena configuración específica	—
APIPago	+	getTipo()	String	Override de categoría	—
APIPago	+	getConfiguracion()	String	Override: añade atributo específico	—
APILogistica	+	APILogistica(String id, String proveedor, String endpoint, boolean conectado, int latenciaMs, String zonaCobertura)	Constructor	Invoca super y almacena configuración específica	—
APILogistica	+	getTipo()	String	Override de categoría	—
APILogistica	+	getConfiguracion()	String	Override: añade atributo específico	—

Clase / interfaz	Visibilidad	Firma / parámetros	Retorno	Propósito y resultado	Validación / manejo
APIFacturacion	+	APIFacturacion(String id, String proveedor, String endpoint, boolean conectado, int latenciaMs, String nitCertificador)	Constructor	Invoca super y almacena configuración específica	—
APIFacturacion	+	getTipo()	String	Override de categoría	—
APIFacturacion	+	getConfiguracion()	String	Override: añade atributo específico	—
Transaccional	+	cobrar(double monto)	String	Contrato de cobro simulado	Monto > 0
Transaccional	+	reintegrar(double monto)	String	Contrato de reintegro simulado	Monto > 0
Documentable	+	generarDocumento(String referencia)	String	Contrato: devuelve XML/guía simulada	—
Auditable	+	consultarAuditoria()	String	Contrato: devuelve auditoría ficticia	—
APIPago	+	cobrar(double monto)	String	Implementa Transaccional	—
APIPago	+	reintegrar(double monto)	String	Implementa Transaccional	—
APIPago	+	consultarAuditoria()	String	Implementa Auditable	—
APILogistica	+	generarDocumento(String referencia)	String	Implementa Documentable: guía	—
APILogistica	+	consultarAuditoria()	String	Implementa Auditable	—
APILogistica	+	rastrear(String guia)	String	Consulta simulada de envío	—
APILogistica	+	cotizar(String destino)	String	Cotización ficticia	—
APIFacturacion	+	generarDocumento(String referencia)	String	Implementa Documentable: XML FEL	—
APIFacturacion	+	consultarAuditoria()	String	Implementa Auditable	—
ControladorIntegraciones	+	ControladorIntegraciones()	Constructor	Inicializa la única lista y carga 10 conexiones	—
ControladorIntegraciones	+	registrar(IntegracionAPI api)	boolean	Parámetro polimórfico: registra sin duplicar id	—
ControladorIntegraciones	+	buscar(String id)	IntegracionAPI	Busca en lista por id; null si no existe	—
ControladorIntegraciones	+	listar()	String	Construye listado sin crear otra lista	—
ControladorIntegraciones	+	ordenarPorLatencia()	void	Collections.sort(integraciones) sobre Comparable	—
ControladorIntegraciones	+	procesarReintegro(Transaccional servicio, double monto)	String	Polimorfismo por interfaz; despacha sin conocer proveedor	—

Clase / interfaz	Visibilidad	Firma / parámetros	Retorno	Propósito y resultado	Validación / manejo
ControladorIntegraciones	+	procesarReintegro(String id, double monto)	String	Overloading: busca y usa instanceof Transaccional	Si no implementa, ignora y retorna mensaje; sin excepción
ControladorIntegraciones	+	procesarCobro(Transaccional servicio, double monto)	String	Polimorfismo por interfaz	—
ControladorIntegraciones	+	procesarCobro(String id, double monto)	String	Overloading por identificador	No compatible: mensaje
ControladorIntegraciones	+	procesarDocumento(Documentable servicio, String referencia)	String	Polimorfismo por interfaz	—
ControladorIntegraciones	+	procesarDocumento(String id, String referencia)	String	Overloading por identificador	No compatible: mensaje
ControladorIntegraciones	+	consultarAuditoria(Auditable servicio)	String	Polimorfismo por interfaz	—
ControladorIntegraciones	+	consultarAuditoria(String id)	String	Overloading por identificador	—
Principal	+ static	main(String[] args)	void	Menú y Scanner; ÚNICA clase con System.out.println	Usar while(opcion != 0), sin while(true)+break

Relaciones UML

Origen	Destino	Relación UML	Java	Multiplicidad	Justificación
APIPago	IntegracionAPI	Generalización	extends	1 → 1	Hereda datos y Comparable
APILogistica	IntegracionAPI	Generalización	extends	1 → 1	Hereda datos y Comparable
APIFacturacion	IntegracionAPI	Generalización	extends	1 → 1	Hereda datos y Comparable
APIPago	Transaccional	Realización	implements	—	Pago sí permite reintegro
APIPago	Auditable	Realización	implements	—	Auditoría
APILogistica	Documentable	Realización	implements	—	Guía de envío
APILogistica	Auditable	Realización	implements	—	Auditoría
APIFacturacion	Documentable	Realización	implements	—	XML FEL
APIFacturacion	Auditable	Realización	implements	—	Auditoría
IntegracionAPI	Comparable<IntegracionAPI>	Realización	implements	—	Orden natural
ControladorIntegraciones	IntegracionAPI	Agregación	ArrayList<IntegracionAPI>	1 → 0..*	ÚNICA colección polimórfica
Principal	ControladorIntegraciones	Dependencia	usa	1 → 1	Salida y entrada centralizada

Preguntas

No.	Pregunta	Respuesta de análisis
1	¿Qué se modela?	Catálogo simulado de integraciones externas de pago, logística y FEL. No se realizan solicitudes HTTP.
2	¿Qué clases son necesarias?	IntegracionAPI abstracta, APIPago, APILogistica, APIFacturacion, ControladorIntegraciones y Principal; interfaces Transaccional, Documentable, Auditable.
3	¿Cómo se aplica herencia?	Tres subclases extienden IntegracionAPI y redefinen getTipo/getConfiguracion.
4	¿Cómo se aplica polimorfismo?	Una ArrayList<IntegracionAPI> contiene todos los subtipos; métodos reciben Transaccional, Documentable o Auditable.
5	¿Qué es overloading?	procesarReintegro(Transaccional,double) y procesarReintegro(String,double), además de cobro, documento y auditoría.
6	¿Qué pasa si se reintegra en logística?	procesarReintegro(String,double) comprueba instanceof Transaccional; si es falso retorna aviso sin llamar ni lanzar excepción.
7	¿Cómo se ordena?	IntegracionAPI implements Comparable<IntegracionAPI>; compareTo usa latenciaMs y desempata por id; Collections.sort ordena la misma lista.
8	¿Qué patrón MVC se aplica?	Modelo: entidades/interfaces; Controlador: catálogo y operaciones; Vista: Principal presenta menú y lee entrada, para cumplir restricción de impresión.
9	¿Qué colección se usa?	Exactamente un campo ArrayList<IntegracionAPI> en ControladorIntegraciones. No se almacenan listas auxiliares.
10	¿Cómo se agregan proveedores nuevos?	registrar(new APILogistica(...)) agrega otra instancia sin cambiar el motor central. Nuevos tipos implementan contratos compatibles.
11	¿Qué datos se cargan?	10 conexiones ficticias: cuatro pagos, tres logísticas, tres facturaciones; estados y latencias simulados.
12	¿Cómo se valida la entrada?	Principal lee cadenas con Scanner.nextLine y convierte números con try/catch NumberFormatException; no interrumpe menú.
13	¿Qué se entrega?	PDF con análisis y UML; enlace al repositorio con código Java y driver program. Excel es material base para el PDF.

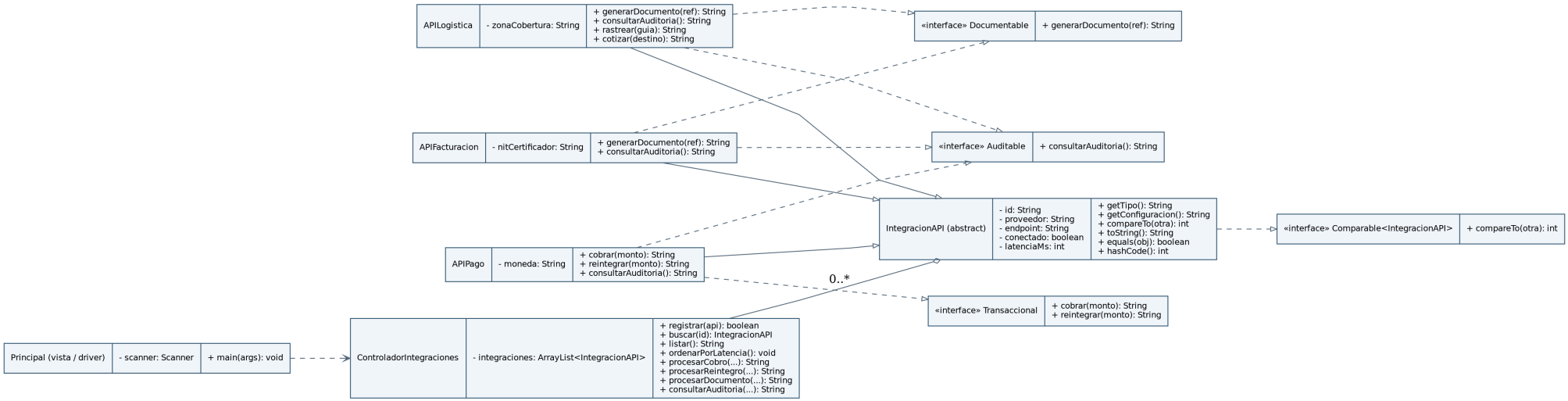
Excepciones

Situación	Condición	Excepción Java	Dónde se controla	Acción de recuperación
ID inexistente	buscar retorna null	Ninguna	ControladorIntegraciones	Devuelve aviso sin colapsar
Reintegro en logística	No implementa Transaccional	Ninguna	ControladorIntegraciones	Ignora petición, devuelve aviso
Documento en pago	No implementa Documentable	Ninguna	ControladorIntegraciones	Devuelve aviso de operación no disponible
Opción de menú inválida	Número fuera del rango	Ninguna	Principal	Muestra aviso y repite
Texto en lugar de número	Integer.parseInt / Double.parseDouble	NumberFormatException	Principal	Captura, muestra aviso y repite
Monto no positivo	monto <= 0	Ninguna	Controlador / Modelo	Devuelve aviso; no realiza operación
ID duplicado al registrar	Ya existe identificador	Ninguna	ControladorIntegraciones	Retorna false y conserva catálogo
Latencia negativa	latenciaMs < 0	IllegalArgumentException	Constructor / setter de IntegracionAPI	Evitar datos inválidos en carga
Cadena vacía	Identificador / referencia vacía	Ninguna	Principal / Controlador	Solicita dato correcto o retorna aviso
Conexión desconectada	conectado == false	Ninguna	Controlador	No simula operación; informa estado
Sin API real	Todo es simulación	Ninguna	Modelo	Retorna mensajes / documentos ficticios

Carga inicial

ID	Proveedor ficticio	Clase	Conectado	Latencia (ms)	Configuración específica
PAG-01	Tarjeta A	APIPago	True	75	GTQ
PAG-02	Tarjeta B	APIPago	True	92	USD
PAG-03	Billetera	APIPago	True	65	GTQ
PAG-04	Transferencia	APIPago	False	145	GTQ
LOG-01	Envíos Express	APILogistica	True	110	Guatemala
LOG-02	Mensajería Local	APILogistica	True	82	Capital
LOG-03	Paquetería Regional	APILogistica	False	165	Centroamérica
FEL-01	Certificador Uno	APIFacturacion	True	125	NIT-FICT-001
FEL-02	Certificador Dos	APIFacturacion	True	100	NIT-FICT-002
FEL-03	Certificador Tres	APIFacturacion	True	130	NIT-FICT-003

Diagrama de clases UML - vista de relaciones



El diagrama editable completo, con constructores, getters, setters y firmas, se incluye en Diagrama_Ejercicio6.drawio.